

Reviewer Pack — Riemann Hypothesis Connection to Resolution Proof Length

Entry 79c0864dc75e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 01:19:36 UTC

Riemann Hypothesis Connection to Resolution Proof Length

Entry ID: 79c0864dc75e **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 01:19:36 UTC

1. Conjecture

Field A (mathematical branch): Number Theory (Riemann Hypothesis) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

[‘For all unsatisfiable CNF formulas F with n variables, the Riemann Zeta function $\zeta(s)$ evaluated at $s = 1/2 + it$ has a zero in the critical strip for some $t \in [0, T_F]$, where T_F is the total length of the resolution proof tree for F .’, ‘This zero exists if and only if F requires more than a constant factor k times as many resolution steps to prove unsatisfiability compared to an average CNF formula of similar size.’, ‘Furthermore, the magnitude of t is inversely proportional to the number of resolution steps.’]

Rationale (proposer’s reasoning):

[‘The Riemann Hypothesis has profound implications in number theory and could potentially provide a new perspective on complexity. If there is a connection between the zeros of the Riemann Zeta function and the length of resolution proofs, it suggests that the proof complexity might be related to deep mathematical structures.’, ‘This conjecture aims to bridge pure mathematics with computational complexity, specifically by exploring the potential use of number theory to analyze the complexity of resolution proofs.’]

Taxonomy category: number_theory_to_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f08f7bd8465a49f2

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all CNF formulas F with $n \leq 40$, there is a zero t in the critical strip $[0, T_F]$ of the Riemann Zeta function $\zeta(s)$ at $s = 1/2 + it$, where T_F is the proof length and this zero is inversely proportional to T_F . The conjecture is falsified if any CNF formula F with $n \leq 40$ does not show such a correlation.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Riemann Hypothesis AND Resolution proof length - critical strip Riemann Zeta function AND resolution proof complexity - inverse proportionality magnitude zero critical strip resolution steps

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.6s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2**n):
            clause = [random.randint(-1, 1) * i for i in range(1, n+1)]
            if sum(clause) != 0:
                clauses.append(clause)
        return clauses

    def resolution(cnf):
        stack = cnf[:]
        while True:
            new_clause = None
            for i in range(len(stack)):
                for j in range(i + 1, len(stack)):
                    if set(stack[i]) & set(stack[j]):
                        new_clause = [x for x in stack[i] if x not in stack[j]] + \
                                    [y for y in stack[j] if y not in stack[i]]
                        break
                if new_clause:
                    break
            if not new_clause:
                return len(cnf) - len(stack)
            stack.append(new_clause)
```

```

def riemann_zeta(s, t):
    epsilon = 1e-6
    result = 0.0
    for k in range(1, int(1 / epsilon)):
        term = (k ** (-s)) * math.exp(-2 * math.pi * k * t)
        if abs(term) < epsilon:
            break
        result += term
    return result

n = random.randint(5, 40)
cnf = generate_cnf(n)
proof_length = resolution(cnf)

s = 0.5 + random.uniform(0, proof_length)
t = math.log(abs(riemann_zeta(s, 0)), abs(riemann_zeta(s, proof_length)))

if t <= proof_length and t >= 0:
    conjecture_holds = True
    counterexample = ""
else:
    conjecture_holds = False
    counterexample = "No zero found in the critical strip for  $\zeta(s)$  at  $s = 1/2 + it$ "

return {
    "metric_name": "Proof Length",
    "metric_value": proof_length,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    total_metric_value = sum(result["metric_value"] for result in results)
    mean_metric_value = total_metric_value / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"No zero found in the critical strip for  $\zeta(s)$  at  $s = 1/2 + it$ \"")
    else:
        print("RESULT: INCONCLUSIVE reason=mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not provide evidence to support or falsify the conjecture. | next: Run the test again with increased time limits and ensure that it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12398
2	preregistration	ollama_remote	glm4:latest	0	0	6690
3	novelty	ollama_remote	glm4:latest	0	0	4653
4	novelty	ollama_remote	glm4:latest	0	0	5493
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16280
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10087
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12962
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9730
9	judge	ollama_remote	glm4:latest	0	0	34682

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 112975 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/79c0864dc75e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/79c0864dc75e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/79c0864dc75e.tar.gz` (if generated)